

Package ‘driftmapR’

September 10, 2026

Title Align Repeated Coordinate Maps and Measure Relative Movement

Version 0.0.3.9000

Description A development prototype for ingesting repeated two-dimensional coordinate maps, aligning them using orthogonal Procrustes transformations, and measuring reference-relative entity movement. Supports changing entity sets, optional alignment anchors, translation, reflection, rotation, and optional isotropic scaling. Provides explicit transformation diagnostics and descriptive trajectory plots. Matches supplied cluster identities by maximum shared-entity overlap, with explicit assignment and ambiguity diagnostics. Includes centered PCA and classical multidimensional scaling adapters with retained inputs, preprocessing, and spectral diagnostics. No inferential uncertainty is estimated.

License MIT + file LICENSE

Encoding UTF-8

Depends R (>= 4.1.0)

Imports ggplot2 (>= 3.4.0), clue (>= 0.3-65), grid, grDevices, stats, utils

Suggests testthat (>= 3.0.0), roxygen2 (>= 7.3.0), knitr, rmarkdown

Config/testthat/edition 3

VignetteBuilder knitr

Roxygen list(markdown = TRUE)

Config/roxygen2/version 8.1.0

NeedsCompilation no

Author Prototype Maintainer [aut, cre]

Maintainer Prototype Maintainer <maintainer@example.org>

Contents

driftmapR-package	2
align_snapshots	2
distance_to_anchor	3
drift_data	4
embed_snapshots	5
match_clusters	7
measure_drift	9
plot_drift_map	10
validate_drift_data	11

Index**12**

driftmapR-package *driftmapR: align repeated maps and measure relative movement*

Description

Ingest user-supplied two-dimensional coordinates with `drift_data()`, align them with `align_snapshots()`, compute descriptive movement with `measure_drift()`, and display the result with `plot_drift_map()`. Match supplied hard cluster labels with `match_clusters()` using entity overlap. Alternatively start from repeated numeric features or labeled distances using `embed_snapshots()` for PCA or classical multidimensional scaling.

Details

The development prototype provides no bootstrap intervals, significance tests, or cluster fitting. A bootstrap specification accompanies the executable embedding adapters; it is not an uncertainty estimator. Movement is relative to the selected alignment entities and coordinate reference.

Author(s)

Maintainer: Prototype Maintainer <maintainer@example.org>

Authors:

- Prototype Maintainer <maintainer@example.org>

align_snapshots *Align coordinate snapshots by orthogonal Procrustes fitting*

Description

Align coordinate snapshots by orthogonal Procrustes fitting

Usage

```
align_snapshots(
  object,
  reference = c("previous", "first"),
  scale = FALSE,
  anchors = NULL,
  rank_tol = 1e-10
)
```

Arguments

object	A driftmap created by <code>drift_data()</code> .
reference	"previous" aligns each raw snapshot to the already aligned previous snapshot; "first" aligns each raw snapshot to the first snapshot.
scale	Estimate an isotropic nonnegative scale factor? Default <code>FALSE</code> preserves coordinate units. Scaling may absorb real expansion/contraction.
anchors	Optional character vector of stable entity IDs used to fit each transformation. Only anchors present in both compared periods are fitted. The transformation is applied to all entities, including new entities.
rank_tol	Relative singular-value tolerance in $(0, 1)$ for rejecting degenerate or nearly degenerate geometry. Default <code>1e-10</code> .

Details

For row-vector coordinates, let centered fitting maps be X (source) and Y (reference). With $X^T Y = U D V^T$, the minimizer is $R = U V^T$. Reflections are allowed: no determinant correction is applied. If enabled, $s = \text{tr}(D) / \|X\|_F^2$; otherwise $s = 1$. Translation is $b = \bar{y} - s \bar{x} R$. Every source point is transformed as $z^* = s z R + b$.

At least three noncollinear fitting entities in both maps, and full-rank cross-covariance, are required for identifiable two-dimensional fitting. Scaling intermediate matrices improves numerical range; singular-value ratios, not a fixed absolute cutoff, define rank. Nonfinite fitted results are rejected. This is pairwise sequential/reference alignment, not generalized consensus Procrustes. Previous-period alignment may accumulate reference drift.

Movement is relative to the fitted reference configuration. Fitting moving entities can absorb genuine shared movement. An entirely coherent global translation/rotation cannot be identified separately from coordinate artifact without external constraints. Anchors encode a substantive stability assumption; they are not discovered or certified by this function.

Value

An updated driftmap. `aligned` contains all transformed rows; `transformations` has one row per time (including an identity first row), with list columns `rotation`, `translation`, `matched_entities`, and `singular_values`, and scalar fit diagnostics. `n_shared` counts all shared entities; `n_matched` counts fitting entities. `rss` is the fitting residual sum of squares; `rmse = sqrt(rss / n_matched)` is a per-entity radial RMSE; `disparity` divides RSS by centered target sum of squares. The first row has zero fitting entities and NA RMSE/disparity. Raw coordinates are kept.

Examples

```
p <- data.frame(entity = letters[1:4], time = 1, x = c(0, 2, 0, 1),
               y = c(0, 0, 2, 1))
q <- transform(p, time = 2, x = -y + 5, y = x - 3)
fit <- drift_data(rbind(p, q)) |> align_snapshots()
fit$transformations[c("time", "reference_time", "n_matched", "rss")]
measure_drift(fit)
```

`distance_to_anchor` *Measure aligned distance to an entity or fixed coordinate anchor*

Description

Measure aligned distance to an entity or fixed coordinate anchor

Usage

```
distance_to_anchor(object, anchor)
```

Arguments

<code>object</code>	An aligned driftmap.
<code>anchor</code>	A character entity ID present in every period, or a finite numeric vector $c(x, y)$ in the aligned reference coordinate system.

Details

This is a distance metric, distinct from the `anchors` used for alignment. An entity anchor may itself move. No distance change or uncertainty is inferred automatically; compare adjacent periods for the same entity.

Value

A tidy data frame with `entity`, `time`, `x`, `y`, `anchor_x`, `anchor_y`, and `distance`. The entity anchor's own distance is zero in every period.

Examples

```
d <- data.frame(entity = rep(letters[1:4], 2), time = rep(1:2, each = 4),
               x = rep(c(0, 2, 0, 1), 2), y = rep(c(0, 0, 2, 1), 2))
fit <- drift_data(d) |> align_snapshots()
distance_to_anchor(fit, "a")
```

`drift_data` *Construct a longitudinal coordinate-map object*

Description

Construct a longitudinal coordinate-map object

Usage

```
drift_data(
  data,
  periods = NULL,
  coords = c("x", "y"),
  original_data = NULL,
  embedding_metadata = list()
)
```

Arguments

<code>data</code>	A data frame with <code>entity</code> , <code>time</code> , and two coordinate columns. Additional columns (for example, supplied cluster labels) are retained.
<code>periods</code>	Optional complete, ordered vector of expected time values. Every declared period must be observed. Numeric, Date, and POSIXct times are sorted by default; ordered factors use their full level order. Character times and unordered factors require explicit <code>periods</code> .
<code>coords</code>	Character vector of exactly two coordinate column names.
<code>original_data</code>	Optional original data retained without interpretation.
<code>embedding_metadata</code>	List of caller-supplied embedding metadata.

Details

Only two-dimensional maps are supported in this prototype. Missing entity observations are allowed, but missing/nonfinite coordinates are not. Without an explicit schedule, numeric gaps cannot identify a missing period: times 1 and 3 are simply two observed snapshots. Supply `periods = 1:3` to detect an entirely missing period 2. No times or coordinates are imputed.

Value

An S3 driftmap list. `coordinates` holds canonical `entity`, `time`, `x`, `y`, and `period_index` columns. `aligned` and `transformations` are populated by `align_snapshots()`. `match_clusters()` populates `clusters`. `movement` and `bootstrap` are reserved slots, initially `NULL`; `movement` functions return tables without mutating the object. `diagnostics` and `settings` record the run.

Examples

```
d <- data.frame(entity = rep(letters[1:4], 2), time = rep(1:2, each = 4),
               x = rep(c(0, 2, 0, 1), 2), y = rep(c(0, 0, 2, 1), 2))
object <- drift_data(d)
print(object)
```

<code>embed_snapshots</code>	<i>Embed repeated feature data or dissimilarities in two dimensions</i>
------------------------------	---

Description

Embed repeated feature data or dissimilarities in two dimensions

Usage

```
embed_snapshots(
  data,
  features = NULL,
  method = c("pca", "cmds"),
  periods = NULL,
  standardize = c("none", "first", "pooled", "period"),
```

```

feature_weights = NULL,
negative_eigen = c("error", "truncate"),
eigen_tol = 1e-10
)

```

Arguments

<code>data</code>	A long data frame containing <code>entity</code> , <code>time</code> , and explicitly selected numeric features. For <code>method = "cmds"</code> , alternatively a list of labeled <code>dist</code> objects or square dissimilarity matrices, one per period.
<code>features</code>	Names of at least two distinct feature columns. Required for data-frame input; never inferred from numeric metadata.
<code>method</code>	"pca" for centered SVD scores or "cmds" for classical MDS.
<code>periods</code>	Ordered schedule, as in <code>drift_data()</code> . For distance lists, supply one value per element, or use fully named lists whose names define the period order. If both are supplied, names must agree with the schedule.
<code>standardize</code>	Standard-deviation reference: "none" (common original units), "first", "pooled" (all entity-time rows, observation weighted), or "period". All feature maps are centered within period. Period-specific scaling can remove substantive changes in feature dispersion.
<code>feature_weights</code>	Optional finite nonnegative feature weights, applied through their square roots. Unnamed weights follow <code>features</code> ; named weights must match them exactly. Weights are not normalized. Integer weights reproduce column multiplicities, including omitted columns with zero weight. These weights alone do not define a bootstrap or uncertainty estimate.
<code>negative_eigen</code>	For classical MDS, "error" rejects materially negative Gram eigenvalues; "truncate" warns and keeps the leading two positive eigenvalues. No additive correction is implemented.
<code>eigen_tol</code>	Relative spectral tolerance strictly between zero and one. Two retained eigenvalues must exceed this fraction of the largest absolute eigenvalue. A second/third eigenvalue tie at this tolerance warns.

Details

Each period needs at least three entities and effective rank two. Feature schemas are common across periods; missing/nonfinite measurements are rejected without imputation. Active constant features cannot be standardized. Inactive features receive scale one. Reserved coordinate names `x` and `y` may be features, but cannot be extra metadata.

For features, the fitted geometry is the centered matrix with column `j` divided by its declared sample SD and multiplied by `sqrt(weight[j])`. The PCA `prcomp` model is fitted to the scaled, weighted input, not directly to raw features. Its scores and loadings, raw centers, and scale/weight vectors are retained. `predict(model, raw_data)` is therefore not a raw-data adapter. Both methods store Gram eigenvalues (squared singular values for PCA). PCA retains the complete SVD spectrum, omitting implicit zero Gram eigenvalues when there are fewer features than entities. PCA component variances additionally remain in `model$sdev^2`.

Distance input needs unique, nonblank entity labels, identical row/column order, finite nonnegative symmetric entries, and an exactly zero diagonal. Feature preprocessing arguments are unavailable

for this input type. MDS diagnostics report material negative eigenvalue counts, negative absolute-inertia fraction (including roundoff), two-dimensional positive and absolute inertia fractions, relative eigenvalue gap, and pairwise distance RMSE. PCA has no negative spectrum and distance RMSE is NA. A tie within the retained plane permits arbitrary rotation; a tie at its boundary makes the selected plane ambiguous. Alignment cannot repair truncation distortion or recover discarded dimensions.

Value

A new, unaligned driftmap. Original input is retained in `original_data`; `embedding_metadata` holds feature order, weights, per-period centers/scales, and fits. `diagnostics$embedding` is a tidy spectrum/reconstruction table. Call `align_snapshots()` before `measure_drift()`. Supplied nonfeature metadata, such as cluster labels, remains in `coordinates` for `match_clusters()`.

See Also

`stats::prcomp()`, `stats::cmdscale()`, `align_snapshots()`

Examples

```
first <- data.frame(entity = letters[1:5], time = 1,
                    a = c(0, 3, 0, 2, 1), b = c(0, 0, 3, 2, 1))
second <- first
second$time <- 2
second$a[5] <- second$a[5] + 0.8
second$b[5] <- second$b[5] + 0.6
fit <- embed_snapshots(rbind(first, second), features = c("a", "b")) |>
  align_snapshots(anchors = letters[1:4])
measure_drift(fit)
fit$diagnostics$embedding
```

match_clusters

Match cluster identities across adjacent snapshots

Description

Match supplied hard cluster labels by maximum shared-entity overlap. This function does not fit clusters and does not require aligned coordinates.

Usage

```
match_clusters(
  object,
  cluster = "cluster",
  min_overlap = 1L,
  min_jaccard = 0,
  noise = NULL,
  diagnose_ties = TRUE
)
```

Arguments

<code>object</code>	A driftmap created by <code>drift_data()</code> .
<code>cluster</code>	Name of the column in <code>object\$coordinates</code> holding supplied cluster labels. Character, factor, and finite numeric labels are supported. Missing labels are unassigned. Blank labels and lossy numeric conversions are rejected. Raw values in <code>coordinates</code> are never replaced.
<code>min_overlap</code>	Minimum number of jointly labeled shared entities for a candidate match; a positive integer. Default 1.
<code>min_jaccard</code>	Minimum Jaccard overlap in $[0, 1]$. Both cluster sizes in this denominator use only entities present and assigned in BOTH snapshots. Eligibility thresholds apply before optimization.
<code>noise</code>	Optional labels to treat as unassigned, for example <code>noise = 0</code> . No numeric label, including zero or minus one, is excluded automatically.
<code>diagnose_ties</code>	Compute an objective-loss margin for each selected link? Default <code>TRUE</code> . Disabling this reduces repeated assignment work but retains deterministic tie resolution; margin/ambiguity diagnostics then are <code>NA</code> .

Details

For adjacent periods, restrict to the common entities whose labels are assigned in both. Let O_{ab} count members in previous cluster a and current cluster b . Maximize $\sum_{ab} O_{ab} M_{ab}$ over binary one-to-one partial matches (row and column sums at most one). Zero overlaps can never continue an identity. Jaccard is $O_{ab}/(O_{a+} + O_{+b} - O_{ab})$. Matching maximizes the sum of overlap COUNTS, not the sum of Jaccards or the number of continued clusters.

Initial labels are ordered by radix sorting of their character values, ignoring factor level order. At each later step previous clusters are ordered by stable ID and current clusters by radix-sorted original label. Among equal maximum-overlap solutions, choose the lexicographically smallest row assignment, placing unmatched after all current labels. Exact integer feasibility comparisons resolve ties without numeric perturbations or dependence on the assignment solver's unspecified tie convention.

Only adjacent periods are linked. A returning label without shared membership receives a new identity. A missing-label/noise observation receives no stable identity. A cluster with no jointly labeled shared members is unmatched; this is not evidence of a real birth/death. Reasons distinguish this lack of evidence from filtering below thresholds or one-to-one competition.

A split candidate means a previous cluster overlaps multiple current clusters; a merge candidate means a current cluster overlaps multiple previous clusters. Flags use all positive flows, including ineligible ones. Ordinary individual switching can produce the same flags. One-to-one matching continues at most one identity through such patterns; this is not a split/merge model, cluster stability estimator, or probability of identity. A positive margin is an objective contrast, not a confidence measure. Results depend on observed common membership and may be unstable with selective entity loss.

Integer assignments use `clue::solve_LSAP()` with zero padding. Independent positive-overlap components are solved separately. Tie diagnostics require additional solves; `diagnose_ties = FALSE` leaves them explicitly unknown.

Value

An updated driftmap. `clusters` is a tidy membership table with `entity`, `time`, `period_index`, `character` `cluster_original`, `stable` `cluster_id`, and `cluster_status` (`initial`, `matched`, `new`, `unassigned`). IDs are strings C0001, C0002, etc., never reused within a run.

`diagnostics$cluster_matching` contains:

registry One row per observed assigned cluster and period, with original/stable labels, total size, current jointly labeled shared size, and status. First-period shared sizes are NA.

assignments Selected links and unmatched previous/current clusters. Includes counts, Jaccard, reasons, and `edge_margin`: the optimal total overlap minus the best total after forbidding that selected link. A zero margin implies another equally optimal correspondence excludes the link. Margins/ambiguity are NA for unmatched rows or disabled diagnostics.

transitions Every positive observed cluster-to-cluster flow, including unselected and threshold-ineligible flows, its overlap count, shares, Jaccard, eligibility, selection, and split/merge candidate flags.

periods Adjacent-pair denominators, cluster counts, matched/new/ unmatched counts, total objective, and pair-level ambiguity.

overlap_matrices, score_matrices Named matrix lists (`p1_to_p2`, etc.). Rows are previous labels in stable-ID order; columns are current labels in radix order. Scores equal overlap counts on eligible edges and zero elsewhere. Zero-size rows/columns for clusters with no shared members are retained. Only assigned labels create rows/columns.

`settings$cluster_matching` records the contract. Coordinates, alignment, and stored geometric movement remain unchanged; stored bootstrap results are cleared because cluster identity has been recomputed.

Examples

```
first <- data.frame(entity = letters[1:6], time = 1,
                   x = c(0, 1, 0, 4, 5, 4), y = c(0, 0, 1, 0, 0, 1),
                   cluster = rep(c("A", "B"), each = 3))

second <- first
second$time <- 2
second$cluster <- rep(c("renamed_2", "renamed_1"), each = 3)
fit <- drift_data(rbind(first, second)) |> match_clusters()
fit$clusters
fit$diagnostics$cluster_matching$assignments
fit$diagnostics$cluster_matching$overlap_matrices[[1]]
```

measure_drift

Measure movement between adjacent aligned snapshots

Description

Measure movement between adjacent aligned snapshots

Usage

```
measure_drift(object)
```

Arguments

`object` An aligned driftmap.

Details

Computes $\Delta_{it} = Z_{it}^* - Z_{i,t-1}^*$ and its Euclidean norm. Gaps are never bridged; entrants/exits have no movement estimate across their unobserved endpoint. Distances use the first map's coordinate units and are not divided by elapsed time. They are descriptive, reference-relative estimates, not statistical evidence of movement. Returned tables do not mutate the input object; callers may assign one to `object$movement`.

Value

A tidy data frame with `entity`, `time_from`, `time_to`, `x_from`, `y_from`, `x_to`, `y_to`, `dx`, `dy`, and `distance`. There is one row per entity observed in both immediately adjacent periods of the stored schedule. Extra input columns are not joined into the movement table.

Examples

```
d <- data.frame(entity = rep(letters[1:4], 2), time = rep(1:2, each = 4),
               x = rep(c(0, 2, 0, 1), 2), y = rep(c(0, 0, 2, 1), 2))
fit <- drift_data(d) |> align_snapshots()
measure_drift(fit)
```

plot_drift_map

Plot repeated positions and adjacent-period trajectories

Description

Display supplied or aligned two-dimensional coordinates. Lines and arrows connect observations only when an entity is present in both immediately adjacent scheduled periods. They never bridge an unobserved period.

Usage

```
plot_drift_map(object, aligned = TRUE, trajectories = TRUE, labels = FALSE)

## S3 method for class 'driftmap'
plot(x, ...)
```

Arguments

<code>object</code>	A driftmap object from <code>drift_data()</code> .
<code>aligned</code>	Logical; use aligned coordinates. The object must first have been processed by <code>align_snapshots()</code> when TRUE.
<code>trajectories</code>	Logical; draw arrows between adjacent observations.
<code>labels</code>	Logical; label entities at their latest observed positions. Overlapping text may be omitted. Dense plots may require a larger plotting device or selective annotation.
<code>x</code>	A driftmap object.
<code>...</code>	Arguments passed to <code>plot_drift_map()</code> .

Details

Colors and point shapes distinguish periods. Equal coordinate-axis scaling preserves angles and relative Euclidean distances. The plot does not fit transformations or estimate uncertainty. Arrows show geometric displacement and are not evidence of statistically significant movement.

With `aligned = FALSE`, raw arrows remain available to diagnose coordinate artifacts and are labeled as `unaligned`. Set `trajectories = FALSE` to show positions alone.

Value

A `ggplot` object that can be customized with `ggplot2`.

Examples

```
first <- data.frame(entity = letters[1:4], time = 1,
                   x = c(0, 2, 0, 2), y = c(0, 0, 1, 1))
second <- transform(first, time = 2, x = -y + 3, y = x - 2)
map <- drift_data(rbind(first, second))
map <- align_snapshots(map)
plot_drift_map(map)
plot_drift_map(map, aligned = FALSE, trajectories = FALSE)
```

`validate_drift_data`

Validate a driftmap object and optional alignment geometry

Description

Validate a driftmap object and optional alignment geometry

Usage

```
validate_drift_data(
  object,
  check_overlap = FALSE,
  reference = c("previous", "first"),
  anchors = NULL,
  rank_tol = 1e-10
)
```

Arguments

<code>object</code>	A driftmap object.
<code>check_overlap</code>	Check matched-entity counts and identifiable two-dimensional geometry for the requested alignment. Construction alone permits sparse maps.
<code>reference</code>	Either "previous" or "first".
<code>anchors</code>	Optional character vector of entity IDs used to fit alignment.
<code>rank_tol</code>	Relative singular-value tolerance, strictly between zero and one.

Details

Alignment requires at least three shared, noncollinear fitting entities and a full-rank cross-covariance. A sufficient count alone is not enough.

Value

The validated object, invisibly; invalid inputs raise informative errors.

Index

*** internal**

driftmapR-package, [2](#)

align_snapshots, [2](#)

align_snapshots(), [2](#), [5-7](#), [10](#)

clue::solve_LSAP(), [8](#)

distance_to_anchor, [3](#)

drift_data, [4](#)

drift_data(), [2](#), [6](#), [7](#), [10](#)

driftmapR(*driftmapR-package*), [2](#)

driftmapR-package, [2](#)

embed_snapshots, [5](#)

embed_snapshots(), [2](#)

match_clusters, [7](#)

match_clusters(), [2](#), [5](#), [6](#)

measure_drift, [9](#)

measure_drift(), [2](#), [6](#)

plot.driftmap(*plot_drift_map*), [10](#)

plot_drift_map, [10](#)

plot_drift_map(), [2](#), [10](#)

stats::cmdscale(), [7](#)

stats::prcomp(), [7](#)

validate_drift_data, [11](#)